MLOps Practical: What is YAML? How to Configure and Use It in MLOps

1. Aim

To understand YAML (YAML Ain't Markup Language), learn its syntax and configuration structure, and implement a YAML configuration file in Python to manage machine learning model parameters in an MLOps workflow.

2. Theory

What is YAML?

YAML stands for YAML Ain't Markup Language. It is a human-readable data serialization language used to store configuration information in a simple, structured format. YAML files commonly use the extensions `.yaml` and `.yml`.

In MLOps, YAML is useful because it separates configuration settings from the main program code. For example, instead of writing the number of decision trees directly inside a Python program, we can store that value in a YAML file and load it when training the model.

YAML is widely used in machine learning projects, CI/CD pipelines, Docker Compose, GitHub Actions, AWS CodeBuild and Kubernetes.

Important YAML syntax rules

- Key-value pairs: Store information using a key and its value.
- Indentation: Spaces define the hierarchy of data. Use consistent spaces, not tabs.
- Lists: Represent multiple values using a hyphen (`-`).
- Nested structures: Organize related configuration settings under a common key.
- Comments: Start with `#`.

Example:

project_name: Tower_Failure_Prediction

model:
  algorithm: RandomForest
  n_estimators: 100
  max_depth: 10

training:
  test_size: 0.2

In this example, `model` contains the model configuration, while `training` contains the training settings. Python can read these values and use them during model training.

Why is YAML important in MLOps?

1. Configuration management: Stores model parameters and dataset paths in one file.
2. Reproducibility: Makes it easier to record and reuse experiment settings.
3. Automation: Defines commands for CI/CD tools such as AWS CodeBuild.
4. Maintainability: Allows parameter changes without modifying the main Python code.
5. Deployment: Supports configuration of containerized and orchestrated applications.

3. Procedure / Steps to Perform the Practical

Step 1: Install the required libraries

Open Command Prompt or terminal and execute:

pip install pyyaml scikit-learn joblib

- `PyYAML` reads YAML files.
- `scikit-learn` trains the machine learning model.
- `joblib` saves the trained model.

Step 2: Create the project folder

Create a folder named `YAML_MLOPS` with the following structure:

```
YAML_MLOPS/
│
├── config.yaml
└── train.py
```

You can create these files using IDLE, Notepad or another code editor.

Step 3: Create the YAML configuration file

Create a file named `config.yaml` and write the following code.

```
project_name: YAML_MLOPS_Practical

model:
  algorithm: RandomForest
  n_estimators: 100
  max_depth: 5
  random_state: 42

training:
  test_size: 0.2
```

Explanation:

- `project_name`: Name of the project.
- `algorithm`: Machine learning algorithm to use.
- `n_estimators`: Number of decision trees in the Random Forest.
- `max_depth`: Maximum depth of each tree.
- `random_state`: Helps make results reproducible.
- `test_size`: Uses 20% of the dataset for testing and 80% for training.

Save the file as `config.yaml`, not `config.yaml.txt`.

Step 4: Create the Python program

Create another file named `train.py` in the same folder.

```
import yaml
import joblib

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 1: Load YAML configuration
with open("config.yaml", "r") as file:
    config = yaml.safe_load(file)

print("Project:", config["project_name"])

# Step 2: Load the dataset
data = load_breast_cancer()
X = data.data
y = data.target
```

```
# Step 3: Split data using YAML settings
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=config["training"]["test_size"],
    random_state=config["model"]["random_state"]
)

# Step 4: Create model using YAML parameters
model = RandomForestClassifier(
    n_estimators=config["model"]["n_estimators"],
    max_depth=config["model"]["max_depth"],
    random_state=config["model"]["random_state"]
)

# Step 5: Train the model
model.fit(X_train, y_train)

# Step 6: Evaluate the model
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("Algorithm:", config["model"]["algorithm"])
print("Number of trees:", config["model"]["n_estimators"])
```

```
print("Test data proportion:", config["training"]["test_size"])

print("Model Accuracy:", round(accuracy * 100, 2), "%")


# Step 7: Save the trained model

joblib.dump(model, "model.pkl")

print("Model saved successfully as model.pkl")
```

Step 5: Execute the program

Open Command Prompt or terminal and navigate to your project folder.

For example, on Windows:

cd Desktop\YAML_MLOPS

Run the program:

python train.py

The program will load the YAML configuration, train a Random Forest classifier using a built-in sample dataset, evaluate its accuracy and save the trained model.

Step 6: Observe the output

The output will look similar to the following. The accuracy may vary depending on the configuration and library version.

```
Project: YAML_MLOPS_Practical
Algorithm: RandomForest
Number of trees: 100
Test data proportion: 0.2
Model Accuracy: 95.61 %
Model saved successfully as model.pkl
```

The program also creates a new file:

```
YAML_MLOPS/
├── config.yaml
```

```
├── train.py
└── model.pkl
```

The `model.pkl` file contains the trained model.

Step 7: Experiment with YAML parameters

Open `config.yaml` and change:

n_estimators: 100

to:

n_estimators: 50

Run the Python program again:

python train.py

The model now uses 50 decision trees instead of 100, without requiring any changes to the Python training code.

This demonstrates a key MLOps principle: separating configuration from code makes machine learning experiments easier to manage and reproduce.

4. Result

The YAML configuration file was successfully created and loaded into Python. The configuration parameters were used to train and evaluate a Random Forest machine learning model, and the trained model was saved as `model.pkl`.